ARCHON

Autonomous, sandbox-verified software repair and AI-stack migration, running on Nebius Token Factory and NVIDIA Nemotron.

NEBIUS TOKEN FACTORY | NVIDIA NEMOTRON 3 | TOKEN FACTORY SANDBOXES | TAVILY SEARCH API

THE PROBLEM

A dependency ships a breaking release. The test suite goes red. Someone reads the traceback, finds the changelog, patches it, re-runs the tests. Chat assistants don't close this loop — they suggest a plausible patch that nobody has executed, against training data that predates the break. Letting an agent actually run code to find out is a real safety problem on a laptop or shared CI runner.

WHAT ARCHON DOES

ARCHON reproduces the failure inside a Nebius Token Factory Sandbox, grounds itself against the live web with Tavily, asks NVIDIA Nemotron 3 to diagnose and patch it, and verifies the fix by re-running the tests — before a human ever sees it.

Reproduce in sandbox → Ground with Tavily → Diagnose & patch (Ultra) → Fork per candidate, test → Review (Super) & present

Each candidate patch runs on its own sandbox fork. Losing attempts are simply never referenced again — there is no rollback, because nothing shared was ever mutated. A second mission type reuses the same loop for AI-stack migration: rewriting OpenAI/Anthropic client code to Nebius Token Factory with Nemotron models, verified by the repo's own tests plus a parity harness.

VERIFIED RESULTS

6 / 6
SWE-bench Verified psf/requests instances resolved in real Token Factory Sandboxes, $0.67 total inference

49s · $0.032
psf__requests-1142: one iteration, reproduce to verified patch

1 iter · $0.016
openai_chat_service migration VERIFIED — client genuinely rewritten, not just green mocks

HOW NEBIUS AND NVIDIA ARE USED

COMPONENT	USED FOR
Nemotron 3 Ultra	Root-cause analysis and patch generation (550B-A55B)
Nemotron 3 Super	Mission supervisor, patch review, Tavily query synthesis (120B-A12B)
Nemotron 3 Nano	Compacting test logs, parsing pass/fail counts (30B-A3B)
Token Factory Sandboxes	Clone, install, run tests, fork one sandbox per candidate patch
Tavily Search API	Live docs/changelog/issue grounding inside the reasoning loop, before the first patch attempt

ARCHITECTURE

Next.js 16 cockpit (mission form, SSE reasoning stream, xterm.js sandbox terminal — one tab per fork, Monaco diff viewer) talks to a FastAPI orchestrator running a deterministic mission state machine. The state machine routes model calls by tier, drives the Token Factory Sandbox client, calls Tavily, and streams every event back to the cockpit live. The public demo (archon-cockpit.vercel.app) is a separate stateless replay server on Vercel implementing the same REST+SSE contract, streaming nine recorded golden-dataset missions with zero credentials and zero spend; live missions run on the self-hosted orchestrator.

ENGINEERING DECISIONS THAT MATTERED

Search-and-replace edits, not unified diffs
Nemotron 3 Ultra's hand-written unified diffs never applied cleanly (0 of 5 live runs). Switching candidates to exact search-and-replace edits, applied by the executor with git diff producing the visible diff, fixed this completely — edits applied first try on every subsequent run.

Structural migration-completeness gate
Our first two live migration missions both came back "VERIFIED" with patches that never touched the client — one was a 2-line test-name change, the other a dependency swap in pyproject.toml. Mocked tests stayed green because the client was never rewritten. We added a post-patch scan that only allows VERIFIED when zero closed-provider client constructions or model literals remain in source.

Strict SWE-bench judging
Resolution is judged only on the instance's FAIL_TO_PASS/PASS_TO_PASS test ids, never on process exit code — the dataset contains truncated ids and test files with unrelated network-dependent tests that would otherwise produce false negatives.

Deterministic supervisor
Mission control flow is plain code, not a model call — Nemotron Super narrates optionally but never decides what happens next. This makes every mission's control flow auditable and reproducible.

KNOWN LIMITATIONS (DOCUMENTED HONESTLY)

Monorepo subdirectory paraphrasing
A subproject buried inside a monorepo resolved 0 of 3 times in Sandboxes, while the identical project as a standalone repo resolved first try. In the longer, multi-file prompt shape, Ultra paraphrased source instead of copying it verbatim — referencing fields and classes that don't exist.

Honest failure: anthropic_summarizer migration
Failed after 8 iterations. The full Messages-API-to-chat-completions translation, plus its Anthropic-shaped test doubles, did not land inside budget — the completeness gate correctly refused every shallow pass rather than reporting a false VERIFIED.

LINKS

LIVE DEMO
archon-cockpit.vercel.app

SOURCE (APACHE 2.0)
github.com/CodeWithEugene/Archon